Prompt me if you can

for PromptKaban

Technical Report

Team members

Giacomo Mazzarella – g.mazzarella@studenti.luiss.it – ID:321931
Arianna Cambi – arianna.cambi@studenti.luiss.it – ID:317971
Sofia Capriolo – sofia.capriolo@studenti.luiss.it – ID:309371
Carmen Gentile – carmenannapi.gentile@studenti.luiss.it – ID:311951

Group delegate: Giacomo Mazzarella

Company

LEAF s.r.l.s.

Luiss Guido Carli
AI Techniques Project
May 2026

Contents

1 Introduction

The goal of this project is to build a semantic prompt search system for LEAF's PromptKaban platform. Given a natural language query, the system should retrieve and rank the most relevant prompts from a large prompt repository.

The challenge focuses on semantic retrieval rather than simple keyword matching, since prompts with similar meanings may use different wording. For this reason, the project aims to develop a pipeline capable of understanding the semantic content of prompts and queries.

2 Methods

2.1 Dataset Loading and Exploratory Analysis

The first step of the pipeline consisted in loading the prompt dataset provided by LEAF and performing a lightweight exploratory data analysis. The raw dataset was stored in JSON format and loaded into a pandas DataFrame through a dedicated loading function. This step also included a validation of the required columns in order to ensure that all the fields needed by the following stages of the pipeline were available.

The exploratory analysis was designed to understand the dataset structure and verify its quality before building the retrieval system. In particular, we inspected the number of rows and columns, the available fields, missing values, duplicated prompt identifiers, and duplicated title-content pairs.

Additional descriptive analyses were performed on the main metadata fields. These included the distribution of categories, the distribution of prompt languages, the frequency of difficulty levels, prompt length statistics, and descriptive summaries of the main engagement and reputation variables, such as likes, upvotes, downvotes, author reputation, views, and uses. We also inspected the distribution of target models associated with the prompts.

These checks were useful to distinguish between fields carrying semantic information and fields better treated as metadata. More specifically, textual fields such as title, category, subcategory, tags, and content were identified as the most informative for semantic retrieval, while numerical and temporal fields were preserved for later stages of the pipeline.

2.2 Preprocessing and text_for_embedding Construction

The preprocessing phase was designed to prepare the dataset for both lexical and semantic retrieval while preserving the original prompt information. Since the following stages of the pipeline rely on semantic representations, aggressive text normalization techniques were intentionally avoided.

Instead of heavily modifying the text, preprocessing focused on creating a unified textual representation for each prompt. A new field called text_for_embedding was introduced by combining the most semantically informative fields of the dataset, namely title, category, subcategory, tags, and content. This combined field provides a richer textual representation of each prompt by including both contextual descriptors and the main prompt body.

This design allows the following retrieval components to capture not only the semantic meaning of the prompt content itself, but also the contextual information given by high-level labels such as

category, subcategory, and tags. In contrast, numerical engagement variables such as likes, upvotes, downvotes, views, and uses, as well as author reputation and creation time, were not included in the textual representation and were preserved separately as metadata.

The processed dataset was finally exported as `processed_prompts.csv`, which represents the shared input for the TF-IDF baseline, the embedding model, and the semantic retrieval components of the project.

2.3 Baseline Models

As a baseline for comparison, we implemented a lexical retrieval system based on TF-IDF and cosine similarity. The baseline was built on top of the text_for_embedding field generated during preprocessing, so that both the lexical and semantic approaches relied on the same textual representation of each prompt.

More specifically, the processed prompt texts were converted into TF-IDF vectors using TfidfVectorizer, and each user query was transformed into the same vector space. Cosine similarity was then computed between the query vector and all prompt vectors in order to rank the prompts by lexical relevance.

The baseline returns the top matching prompts together with their metadata and similarity scores. This component provides a simple and interpretable benchmark against which the semantic retrieval pipeline can be compared in later stages of the project.

2.4 Embedding Model

To support semantic retrieval, prompts and queries were encoded into dense vector representations using the model the model `sentence-transformers/all-MiniLM-L6-v2`. This model was selected because it provides compact sentence embeddings, is lightweight enough for a student project, and is widely used for semantic similarity tasks.

The same embedding model was used for both the prompt texts and the input queries, so that they could be represented in the same semantic vector space. In practice, the model was applied to the text_for_embedding field created during preprocessing, which combines the most semantically informative textual fields of each prompt.

In addition, the generated embeddings were normalized. This design is consistent with cosine-based retrieval, since normalized vectors make similarity comparisons more stable and interpretable. Each prompt was therefore represented by a dense embedding vector of fixed dimension, which was later stored in the vector database and used for semantic search.

2.5 Vector Database

Once the prompt embeddings were generated, they were stored in a persistent ChromaDB collection. ChromaDB was selected as the vector database because it offers a simple Python-native interface, supports persistent local storage, and is well suited for educational semantic search pipelines.

The vector store was configured to use cosine distance as the retrieval metric, in line with the semantic retrieval design of the project. Each prompt embedding was stored together with its identifier and the associated metadata needed for later interpretation and possible ranking extensions.

More specifically, the stored metadata included: id, title, content, category, subcategory, tags, likes, upvotes, downvotes, author_reputation, and created_at. This choice allows the system not only to retrieve semantically similar prompts, but also to preserve the contextual and descriptive information required to inspect and compare results.

The vector store was built from the processed dataset and populated with one embedding per prompt. As a result, the final ChromaDB collection contains the full prompt repository in vector form and can be queried efficiently during the retrieval stage.

2.6 Retrieval

The retrieval stage takes a natural language query as input and returns the most relevant prompts from the vector database. The query is first encoded using the same sentence-transformers model adopted for the prompts, so that the query vector lies in the same semantic space as the stored prompt embeddings.

The resulting query embedding is then used to search the ChromaDB collection. The vector database returns the top candidate prompts according to cosine-based similarity. Since ChromaDB exposes cosine distance values, these were converted into a more intuitive similarity_score by applying the transformation 1 - distance. In this way, higher values correspond to stronger semantic similarity between the query and the retrieved prompt.

The retrieval output is structured as a ranked list of prompt results. For each result, the system returns the prompt identifier, title, content, category, similarity score, and the selected metadata fields such as likes, upvotes, downvotes, author reputation, and creation time. This makes the retrieval stage directly interpretable and prepares the system for future reranking or metadata-aware ranking extensions.

2.7 Reranking Strategy

Describe the reranking component.

2.8 Metadata-Aware Ranking

Describe how metadata information was integrated into the ranking process.

2.9 Evaluation Metrics

Describe the evaluation framework and justify the selected metrics.

Metrics may include:

- Precision@K;
- Recall@K;
- Mean Reciprocal Rank (MRR);
- latency measurements.

3 Results and Discussion

3.1 Dataset and Preprocessing Results

The exploratory analysis confirmed that the dataset was already well structured and suitable for retrieval experiments. The collection contains 20,000 prompts and 20 original fields. No missing values were found, and prompt identifiers were unique across the dataset. A small number of duplicated title-content pairs was detected (93 cases), but these were reported rather than removed, since the dataset was treated as the original input provided for the project.

The analysis also showed that the dataset is distributed across multiple categories without a single dominant class. The largest categories account for only a limited share of the total collection, which suggests that the repository covers a broad range of prompt use cases. In addition, the dataset is almost entirely in English, making it consistent with the embedding model selected for the semantic search stage.

From the metadata analysis, engagement-related variables such as likes, upvotes, downvotes, views, and uses showed high variability across prompts. Author reputation also displayed a broad distribution. These fields were therefore preserved as metadata rather than injected into the textual representation used for retrieval.

The preprocessing stage produced a new field called text_for_embedding, built by combining title, category, subcategory, tags, and content. This field was designed to provide a richer textual representation of each prompt by merging both contextual descriptors and the original prompt body. The resulting processed dataset was exported as `processed_prompts.csv`, with 20,000 rows and 21 columns, and used as the shared input for both the TF-IDF baseline and the semantic retrieval pipeline.

3.2 Technical Results

At the current stage of the project, the implemented technical results concern dataset preparation, lexical baseline retrieval, embedding generation, vector storage, and semantic retrieval.

From the preprocessing stage, the project produced a processed dataset containing 20,000 prompts and 21 columns, including the newly constructed `text_for_embedding` field. This field represents the textual input used by both the TF-IDF baseline and the semantic retrieval pipeline.

For the lexical baseline, the processed prompt texts were converted into TF-IDF vectors and queried through cosine similarity. This baseline provides an interpretable retrieval system that can be used as a reference point for later comparisons.

For the semantic component, dense embeddings were generated using the model `sentence-transformers/all-Min`. Each prompt was encoded into a normalized dense vector representation, and the resulting embeddings were stored in a persistent ChromaDB collection configured for cosine-based retrieval. The final vector store contains 20,000 indexed prompts, confirming that the semantic indexing stage was completed successfully.

At query time, the system encodes the input query with the same embedding model and retrieves the most relevant prompts from the vector database. The returned cosine distance values are converted into an interpretable `similarity_score`, so that higher values correspond to stronger

semantic similarity between the query and the retrieved prompt. The current retrieval output includes both semantic scores and the selected metadata fields associated with each prompt.

A simple end-to-end comparison between TF-IDF retrieval and semantic retrieval was also implemented through the main execution script. This makes it possible to inspect the behavior of the two approaches side by side for the same input query.

This section will be extended once Giacomo's part is completed, in order to include the reranking stage, metadata-aware ranking, and the corresponding full retrieval results.

3.3 Comparative Analysis

At the current stage, the comparison can only be performed between the two implemented retrieval approaches: the TF-IDF lexical baseline and the semantic retrieval pipeline based on dense embeddings and ChromaDB.

The TF-IDF baseline performs well when the user query shares explicit lexical overlap with the target prompts. In these cases, the system tends to retrieve prompts containing the same or very similar words as those appearing in the input query. This makes the baseline simple, transparent, and useful as a reference system. However, it is strongly dependent on surface wording and may favor short or literal matches even when they are less informative.

The semantic retrieval pipeline shows a different behavior. By comparing dense vector representations rather than only word overlap, it is able to retrieve prompts that are closer to the intended meaning of the query. In practice, this leads to results that are often more complete and semantically aligned with the user's intent, especially when the query is written as a natural language sentence.

This difference is visible in the current query example used for testing, namely *"Write a speech to ask for a salary raise"*. The TF-IDF baseline tends to reward prompts with strong lexical overlap around terms such as *speech*, *salary*, and *raise*, while semantic retrieval is more likely to return prompts that better capture the professional and persuasive intent behind the request.

At this stage, the comparison remains qualitative rather than quantitative, since the full evaluation framework has not yet been implemented. Nevertheless, the current outputs already show that TF-IDF provides a useful lexical benchmark, while semantic retrieval offers a more meaning-oriented ranking of prompts.

This section will be completed once Giacomo's part is available, so that the comparison can also include reranking, metadata-aware ranking, and the final quantitative evaluation across all methods.

3.4 Project-Oriented Results

Discuss the potential business value of the proposed system for PromptKaban.

Possible benefits:

- improved prompt discoverability;
- better user experience;
- increased prompt reuse;
- support for community-driven prompt engineering;
- scalability for large prompt repositories.

4 Conclusions

Summarize the main findings of the project.

Highlight:

- main technical contributions;
- effectiveness of semantic retrieval;
- benefits of reranking;
- usefulness of metadata-aware scoring.

Discuss possible future work, such as:

- fine-tuning embedding models;
- learning-to-rank approaches;
- larger-scale evaluation;
- integration into production systems;
- user feedback optimization.